MODELARE ȘI SIMULARE

Săptămâna 2. Aleatorul în simulare și metoda Monte Carlo

Suport de curs · Anul III, Calculatoare și Tehnologia Informației
Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să alegeți o distribuție potrivită pentru o mărime aleatoare (sosiri, durate, defecte);
2. să calculați media și dispersia unei distribuții simple;
3. să explicați cum funcționează un generator pseudo-aleator și rolul seed-ului;
4. să generați variabile aleatoare prin transformarea inversă;
5. să estimați o mărime prin metoda Monte Carlo și să raportați un interval de încredere;
6. să estimați câte repetări sunt necesare pentru o precizie dată.

Cuvinte cheie: *variabilă aleatoare, distribuție, funcție de repartiție, medie, dispersie, Poisson, exponențială, normală, generator pseudo-aleator, seed, transformare inversă, Monte Carlo, interval de încredere, reducerea variației*

1. Probabilități strict necesare

1.1. De ce avem nevoie de probabilități

Intrările unui sistem real sunt incerte: nu știm când sosește următorul client, cât durează o cerere sau când se defectează un disc. Nu putem prezice o valoare anume, dar putem descrie cât de probabile sunt diversele valori – această descriere se numește **distribuție de probabilitate**. O simulare stocastică primește distribuții la intrare (timpi între sosiri, durate de servire, defecte), le combină după regulile modelului și, după multe rulări, produce **distribuția ieșirii** (de exemplu a timpului de așteptare), din care raportăm medii, intervale și percentile (Holban, cap. 9).

1.2. Probabilitate și variabilă aleatoare

Un **experiment aleator** este un experiment al cărui rezultat nu poate fi prezis exact (aruncarea unei monede, sosirea unui client). **Probabilitatea** unui eveniment este un număr între 0 și 1; interpretarea cea mai utilă în simulare este cea **frecvențială**: probabilitatea este frecvența relativă a evenimentului atunci când repetăm experimentul de foarte multe ori. **Legea numerelor mari** garantează că frecvența relativă converge către probabilitate – dar, cum vom vedea, convergența este lentă. Într-o simulare cu 2 000 de aruncări de monedă, frecvența relativă a „capului” a fost 0,7 după 10 aruncări și 0,505 după 2 000.

O **variabilă aleatoare** este o mărime numerică al cărei rezultat depinde de un experiment aleator: numărul de clienți care sosesc într-un minut, durata unei cereri, numărul de calculatoare vândute într-o săptămână. O variabilă este **discretă** dacă ia valori numărabile (0, 1, 2, …) și **continuă** dacă ia valori reale într-un interval.

1.3. Distribuție, funcție de repartiție, medie, dispersie

O variabilă discretă se descrie prin **funcția de masă** $p(x) = P(X = x)$: toate valorile sunt între 0 și 1 și suma lor este 1. Tabelul 1 prezintă un exemplu din cursul profesorului Holban: vânzările săptămânale de calculatoare ale unei firme, observate timp de 30 de săptămâni. Probabilitățile sunt frecvențele relative – este o **distribuție empirică**.

Tabelul 1. Distribuția vânzărilor săptămânale de calculatoare (Holban, cap. 9)

Vânzări x	Nr. de săptămâni	P(X = x)	F(x) = P(X ≤ x)
0	6	0,20	0,20
1	12	0,40	0,60
2	6	0,20	0,80
3	3	0,10	0,90
4	3	0,10	1,00

Funcția de repartiție (CDF) F(x) = P(X ≤ x) dă probabilitatea ca variabila să fie cel mult x; ea crește de la 0 la 1, în trepte pentru variabile discrete, iar P(a < X ≤ b) = F(b) − F(a). O variabilă continuă se descrie printr-o **densitate** f(x): probabilitatea unui interval este **aria** de sub densitate pe acel interval, aria totală este 1, iar probabilitatea oricărei valori exacte este **zero**. Densitatea nu este o probabilitate și poate depăși 1.

Media (valoarea așteptată) este E[X] = Σ x·P(X = x) pentru variabile discrete și E[X] = ∫ x f(x) dx pentru variabile continue. **Dispersia** Var(X) = E[(X − μ)²] = E[X²] − μ² măsoară împrăștierea în jurul mediei; **abaterea standard** σ = √Var(X) are aceeași unitate ca X; **coeficientul de variație** CV = σ/μ măsoară variabilitatea relativă. Pentru vânzări:

$$E[X] = 0\cdot0{,}2 + 1\cdot0{,}4 + 2\cdot0{,}2 + 3\cdot0{,}1 + 4\cdot0{,}1 = 1{,}5$$

E[X²] = 3,7, deci Var(X) = 3,7 − 1,5² = 1,45 și σ ≈ 1,204 calculatoare. Variabilitatea contează: două servere cu același timp mediu de răspuns pot oferi experiențe foarte diferite dacă unul are răspunsuri constante și celălalt răspunsuri foarte variabile. În cozi, variabilitatea creează așteptare chiar dacă media este bună (săptămâna 4), de aceea raportăm și percentile (p95, p99), nu doar media.

De reținut

Media ponderează fiecare valoare cu probabilitatea ei (nu este media aritmetică a valorilor). Pentru o variabilă continuă, P(X = x) = 0; au sens doar probabilitățile intervalelor. Funcția de repartiție F caracterizează complet o distribuție și este instrumentul folosit pentru generarea variabilelor.

1.4. Distribuțiile de lucru ale simulării

Uniforma U(a, b) are densitatea 1/(b − a) pe [a, b], media (a + b)/2 și dispersia (b − a)²/12. O folosim când știm doar limitele unei mărimi și, mai ales, ca materie primă: U(0, 1) este ce produce un generator de numere aleatoare.

Bernoulli(p) descrie o încercare cu două rezultate (1 cu probabilitatea p, altfel 0). **Binomiala Bin(n, p)** numără succesele din n încercări independente: P(X = k) = C(n, k)·p^k·$(1 − p)^{n−k}$, cu media np și dispersia np(1 − p). De exemplu, dacă fiecare din 100 de cereri eșuează independent cu p = 0,02, ne așteptăm în medie la 2 eșecuri.

Poisson(λ) numără evenimentele rare și independente dintr-un interval: P(X = k) = e^(−λ)·$λ^k$/k!, iar media și dispersia sunt ambele egale cu λ. Pentru un server cu λ = 3 cereri pe secundă, P(0 cereri) = e^{-3} ≈ 0,05, iar P(X ≥ 7) ≈ 0,03: rar, dar suficient de des ca să creeze cozi dacă serverul procesează doar 3 cereri pe secundă. Binomiala cu n mare și p mic este bine aproximată de Poisson(np).

Exponențiala Exp(λ) descrie timpul până la următorul eveniment: f(x) = λe^(−λx), F(x) = 1 − e^(−λx), media 1/λ și abaterea standard tot 1/λ (CV = 1). Este singura distribuție continuă **fără memorie**: P(X > t + s | X > t) = P(X > s) – dacă am așteptat deja 5 minute, timpul rămas are aceeași distribuție ca la început. Este potrivită pentru sosiri independente și defecte aleatoare, dar nu pentru durate cu uzură sau cu un minim.

Poisson și exponențiala sunt două fețe ale **procesului Poisson**: dacă evenimentele sosesc independent cu rata constantă λ, numărul lor într-un interval de lungime t este Poisson(λt), iar timpii dintre ele sunt Exp(λ). La λ = 2 sosiri pe minut avem în medie 120 de sosiri pe oră, 30 de secunde între sosiri și P(nicio sosire într-un minut) = $e^{-2} \approx 0{,}135$.

Normala N(μ, σ²) este curba în formă de clopot; aproximativ 68%, 95% și 99,7% din valori se află la cel mult 1, 2 și 3 abateri standard de medie (mai precis, 95% între μ − 1,96σ și μ + 1,96σ). Apare când o mărime este suma multor efecte mici independente și, foarte important, pentru **mediile** rezultatelor unei simulări. Standardizarea Z = (X − μ)/σ transformă orice normală în N(0, 1).

Multe mărimi din informatică au **cozi grele**: valori extreme mult mai frecvente decât prevede normala (dimensiunea fișierelor, durata sarcinilor, latența rețelei). Modele tipice sunt lognormala și distribuția Pareto. Înlocuirea lor cu o exponențială „pentru simplitate” face ca simularea să subestimeze cozile. Când nicio formulă nu se potrivește, folosim o **distribuție empirică**, construită direct din date; ea reproduce fidel datele, dar nu poate genera valori mai extreme decât cele observate.

Tabelul 2. Orientare pentru alegerea distribuției

Mărimea modelată	Distribuție	Parametri
Știu doar un minim și un maxim	uniformă	a, b
Știu minim, cel mai probabil, maxim	triunghiulară	a, m, b
Succes / eșec al unei încercări	Bernoulli	p
Număr de evenimente într-un interval	Poisson	λ
Timp între evenimente independente	exponențială	λ (sau media 1/λ)
Sumă / medie a multor efecte	normală	μ, σ
Multe date, nicio formulă potrivită	empirică	datele

1.5. Populație, eșantion, histogramă

În practică nu cunoaștem distribuția adevărată (populația), ci doar un **eșantion** de observații (Holban, cap. 4). Din eșantion calculăm **media de selecție** $\bar{x} = (1/n)\Sigma x_i$ și **dispersia de selecție** $s^2 = \Sigma(x_i - \bar{x})^2/(n - 1)$, care estimează μ și σ². Împărțirea la n − 1 (corecția Bessel) face estimarea nedeplasată; în NumPy se scrie np.std(x, ddof=1). Eșantionul trebuie să fie reprezentativ, adică ales aleator. **Histograma** – frecvențele relative pe intervale – este felul în care vedem o distribuție din date (Figura 1).

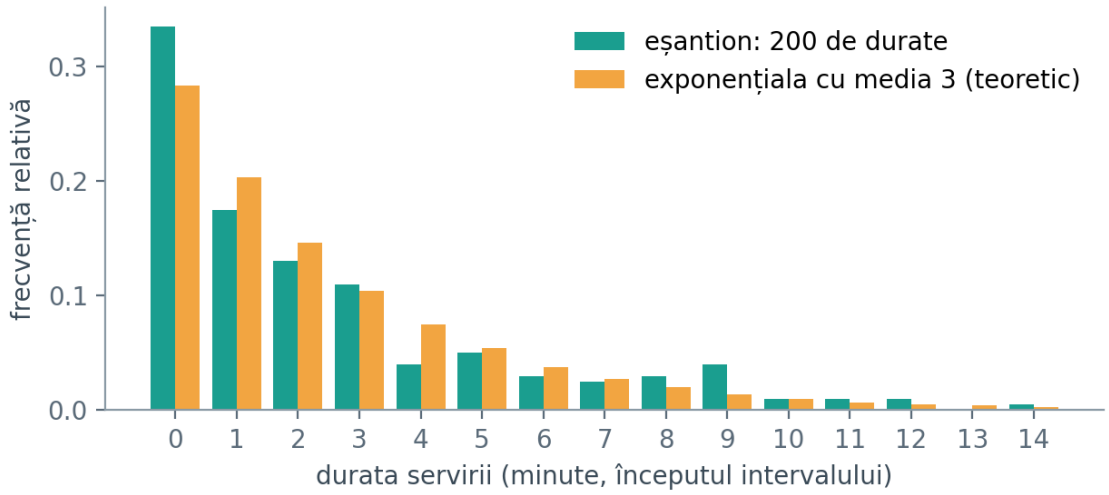


Figura 1. Histograma a 200 de durate de servire generate dintr-o exponențială cu media 3 minute, comparată cu probabilitățile teoretice (media de selecție: 3,02; abaterea standard: 3,05)

De reținut

Poisson numără evenimente, exponențiala măsoară timpul dintre ele. Media și dispersia Poisson sunt egale cu λ. Exponențiala nu are memorie. Mediile rezultatelor unei simulări sunt aproximativ normale. Dispersia de selecție se calculează cu n − 1.

2. De unde vine aleatorul în calculator

2.1. Aleator adevărat și pseudo-aleator

Numerele **aleatoare adevărate** provin din fenomene fizice (zgomot termic, radioactivitate) și sunt disponibile prin /dev/random sau instrucțiunea RDRAND; sunt imprevizibile, dar lente și nereproductibile, și se folosesc în criptografie. Numerele **pseudo-aleatoare** sunt produse de un algoritm determinist pornit dintr-o valoare inițială numită **seed**; sunt rapide și reproductibile și se folosesc în simulare. Von Neumann glumea că oricine produce cifre aleatoare prin metode aritmetice se află „în stare de păcat” – totuși, pentru simulare ajung secvențe care trec toate testele statistice.

De la un generator pentru simulare cerem: **uniformitate** și **independență** (cele două proprietăți statistice fundamentale), **perioadă** foarte lungă, **viteză**, **reproductibilitate** și posibilitatea de a produce **fluxuri independente** pentru rulări paralele (Gibbens, Cambridge).

2.2. Generatorul congruențial liniar

Cel mai simplu generator este **LCG**: $X_{n+1} = (a \cdot X_n + c) \bmod m$, $U_n = X_n/m$. Perioada este cel mult m. Cu c = 0 obținem generatorul multiplicativ; varianta clasică „minimal standard” folosește $m = 2^{31} - 1$ și a = 16 807, cu o perioadă de aproximativ $2{,}1 \cdot 10^9$ – epuizată în câteva secunde de un procesor modern. Exemplu de jucărie: cu a = 5, c = 3, m = 16 și $X_0 = 7$ obținem 6, 1, 8, 11, 10, 5, 12, 15, 14, 9, 0, 3, 2, 13, 4, 7 – toate cele 16 valori, deci perioada maximă. Dacă alegem c = 0 (cu $X_0 = 1$), secvența devine 1, 5, 9, 13, 1, … – perioada scade la 4.

Calitatea depinde enorm de parametri. Figura 2 arată perechi de numere consecutive (U_n, U_{n+1}): pentru un LCG cu modul mic punctele cad pe drepte paralele, deci numerele consecutive nu sunt independente.

Celebrul generator **RANDU** (IBM, anii 1960, $X_{n+1} = 65\,539 \cdot X_n \bmod 2^{31}$) avea tripletele consecutive pe doar 15 plane în cubul unitate, iar rezultatele multor simulări din acea perioadă au devenit suspecte.

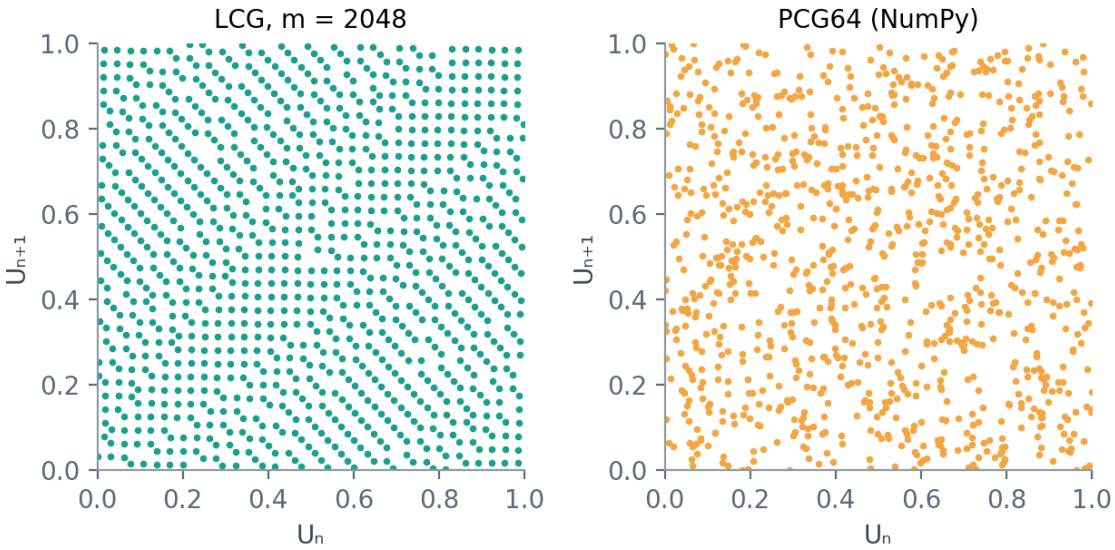


Figura 2. Perechi de numere consecutive: un LCG cu modul mic (stânga) și PCG64 din NumPy (dreapta)

Astăzi folosim generatoare testate: **Mersenne Twister** (1998, perioada $2^{19937} - 1$, folosit de modulul random din Python) și **PCG64** (2014, perioada 2^{128}, generatorul implicit al np.random.default_rng()). Generatoarele se testează cu histograme și testul χ^2 (uniformitate), cu autocorelații și grafice de perechi (independență) și cu baterii întregi de teste precum TestU01.

Simulare ≠ securitate. Generatoarele pentru simulare sunt previzibile: după 624 de ieșiri consecutive ale lui Mersenne Twister, starea poate fi reconstruită și toate valorile viitoare devin cunoscute. Pentru parole, token-uri și chei se folosesc generatoare criptografice (modulul **secrets** sau os.urandom). Invers, în simulare nu folosim generatoare criptografice, fiindcă sunt mai lente și nereproductibile.

2.3. Seed, reproductibilitate, fluxuri independente

Același seed produce exact aceeași secvență, deci exact aceeași simulare – esențial pentru depanare și pentru reproductibilitate. Seed-ul se fixează **o singură dată**, la începutul simulării; dacă este resetat în fiecare pas al buclei, toate „aruncările” devin identice, iar aleatorul dispare fără ca programul să dea vreo eroare. Pentru replicări folosim **fluxuri independente**, create corect din același seed principal:

```
import numpy as np

rng = np.random.default_rng(42)          # PCG64, recomandat
ss = np.random.SeedSequence(2026)        # seed principal
fluxuri = [np.random.default_rng(s) for s in ss.spawn(30)]

def o_replicare(rng):
    return rng.exponential(0.5, size=1000).mean()

rezultate = [o_replicare(r) for r in fluxuri]   # serie sau paralel
```

De reținut

Folosiți np.random.default_rng(seed); fixați seed-ul o singură dată; notați-l în raport; folosiți SeedSequence.spawn pentru replicări; pentru securitate folosiți secrets, nu random.

2.4. De la U(0, 1) la orice distribuție

Transformarea inversă. Dacă U ~ U(0, 1), atunci X = F^{-1}(U) are funcția de repartiție F, deoarece P(X ≤ x) = P(U ≤ F(x)) = F(x). Intuitiv, alegem U pe axa verticală a graficului lui F și citim x pe axa orizontală. Pentru **exponențială**, din U = 1 − e^(−λX) rezultă X = −ln(1 − U)/λ, iar pentru că 1 − U este tot uniformă putem folosi X = −ln(U)/λ. De exemplu, cu λ = 2 și U = 0,5 obținem X ≈ 0,347 minute.

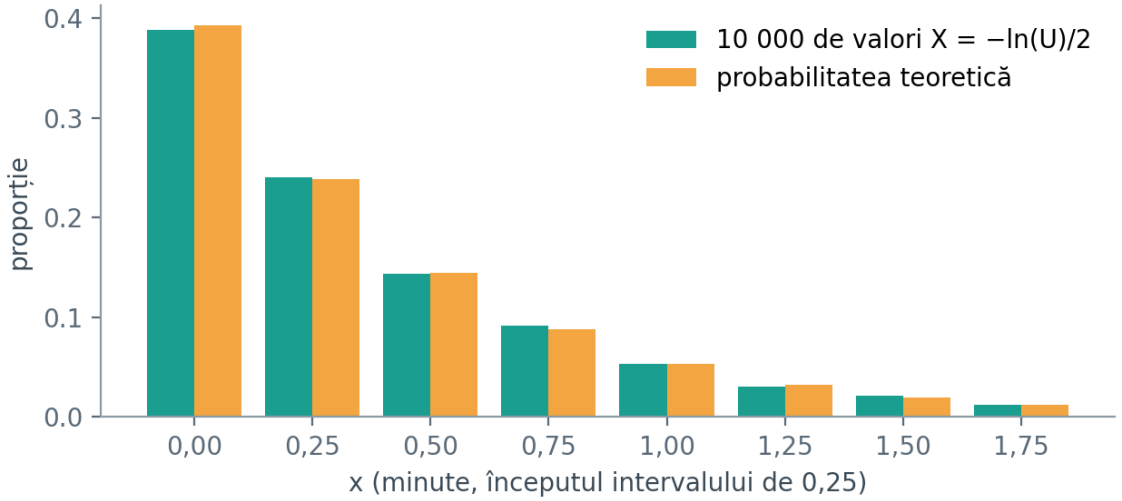


Figura 3. Histograma a 10 000 de valori generate prin X = −ln(U)/2, comparată cu probabilitățile teoretice (media obținută: 0,503, teoretic 0,5)

Pentru **uniforma pe [a, b]**, X = a + (b − a)U; pentru un întreg uniform între a și b, K = ⌊a + (b − a + 1)U⌋ (cu U = 0,709084 și intervalul [6, 10] obținem 9, ca în exemplul lui Holban). Pentru o **distribuție discretă** oarecare, împărțim intervalul [0, 1) în bucăți cu lungimile egale cu probabilitățile și alegem cea mai mică valoare x pentru care U < F(x). Cu tabelul vânzărilor, U = 0,65 cade în [0,6; 0,8) și generează x = 2. Tabelul 3 reproduce simularea a 10 săptămâni din Holban (tabelul 9.5).

Tabelul 3. Simularea a 10 săptămâni de vânzări cu transformarea inversă (media simulată 1,5, abaterea standard 1,269)

Săpt.	U	Vânzări
1	0,486281	1
2	0,928927	4
3	0,205136	1
4	0,852845	3
5	0,005645	0
6	0,651622	2
7	0,799931	2
8	0,333693	1

Săpt.	U	Vânzări
9	0,416841	1
10	0,011913	0

Acceptare – respingere. Când F^{-1} nu are formulă, alegem o anvelopă $c \cdot g(x) \geq f(x)$, generăm Y din g și U uniform și acceptăm Y dacă $U \cdot c \cdot g(Y) \leq f(Y)$; altfel repetăm. Proporția de acceptare este 1/c. Pentru $f(x) = 6x(1 - x)$ pe [0, 1], cu anvelopa uniformă de înălțime c = 1,5, am acceptat 66,7% din propuneri, iar media valorilor acceptate a fost 0,499 (teoretic 0,5).

Normala nu are o inversă a CDF-ului în formă închisă; transformarea **Box–Muller** (1958) produce două normale standard independente din două uniforme: $Z_1 = \sqrt{(-2 \ln U_1)} \cdot \cos(2\pi U_2)$, $Z_2 = \sqrt{(-2 \ln U_1)} \cdot \sin(2\pi U_2)$, iar $X = \mu + \sigma Z$. În practică, NumPy are toate aceste distribuții gata implementate (rng.uniform, rng.integers, rng.exponential – atenție, parametrul este scale = 1/λ –, rng.poisson, rng.normal, rng.binomial, rng.triangular, rng.lognormal, rng.choice). Principiul ne trebuie pentru distribuții proprii (de exemplu empirice) și pentru depanare.

Procesul Poisson se generează adunând timpi exponențiali: pornim de la t = 0, adăugăm Exp(λ) până depășim orizontul T și reținem momentele sosirilor. Cu λ = 2 sosiri pe minut și T = 10 minute ne așteptăm în medie la 20 de sosiri; într-o rulare cu seed 8 au fost 28. Sosirile nu sunt uniform distanțate: pauzele lungi și „ciorchinii" sunt normali și sunt sursa principală a cozilor.

3. Metoda Monte Carlo

3.1. Ideea și istoria

Metoda **Monte Carlo** rezolvă o problemă numerică prin eșantionare aleatoare repetată: generăm multe scenarii aleatoare, calculăm rezultatul fiecăruia și îl analizăm statistic. Holban distinge două sensuri: calculul unor mărimi deterministe (integrale, arii, π) cu ajutorul aleatorului și înlocuirea unui fenomen real cu un experiment statistic (riscul unui proiect, vânzări). Ideea apare la Buffon (1777): un ac de lungime l aruncat pe o podea cu linii paralele la distanța d ≥ l taie o linie cu probabilitatea $2l/(\pi d)$, deci numărând intersecțiile putem estima π. Metoda modernă începe în 1946 cu Stanislaw Ulam, care se întreba cât de des reușește un pasians; împreună cu von Neumann și Metropolis a aplicat-o la difuzia neutronilor pe ENIAC, iar numele apare în articolul „The Monte Carlo Method" (Metropolis & Ulam, 1949).

Rețeta are cinci pași (Holban, cap. 9.2): (1) construim modelul $y = f(x_1, \ldots, x_q)$; (2) generăm un scenariu de intrări din distribuțiile lor; (3) evaluăm modelul și memorăm y_i; (4) repetăm pentru i = 1, …, N; (5) analizăm rezultatele – medie, histogramă, percentile și **interval de încredere**.

3.2. Estimarea lui π și legea erorii

Aruncăm N puncte uniform în pătratul $[0, 1]^2$, iar k dintre ele cad în sfertul de cerc $x^2 + y^2 \leq 1$. Raportul ariilor este π/4, deci $\pi \approx 4k/N$. Cu 1 000 de puncte, 780 au căzut în cerc și am obținut $\pi \approx 3{,}12$ (Figura 4).

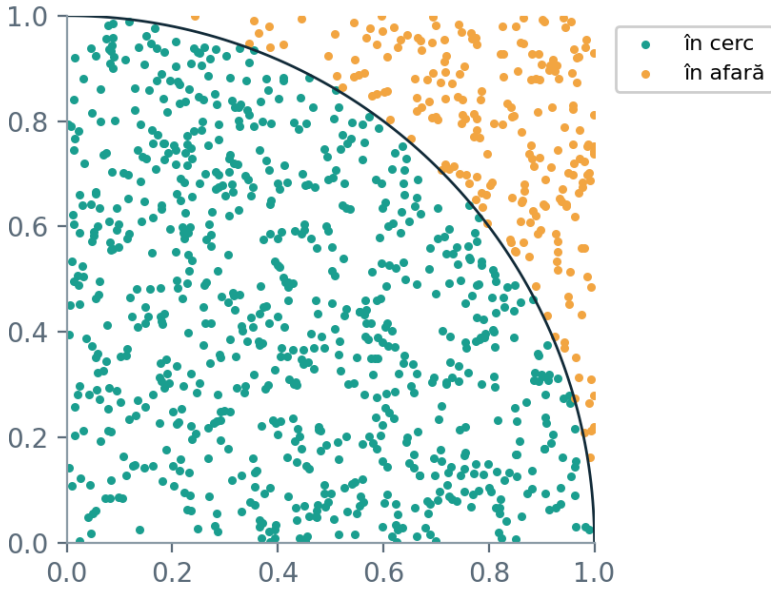


Figura 4. 1 000 de puncte aleatoare în pătratul unitate; estimarea obținută: π ≈ 3,12

```
import numpy as np

def estimeaza_pi(N, rng):
    x, y = rng.random(N), rng.random(N)
    return 4 * np.count_nonzero(x**2 + y**2 <= 1) / N

rng = np.random.default_rng(2026)
print(estimeaza_pi(10**6, rng))
```

Repetând estimarea de 200 de ori pentru fiecare N, eroarea tipică scade exact ca **σ/√N** (Figura 5): fiecare înmulțire a lui N cu 10 împarte eroarea la √10 ≈ 3,16. Aceasta este **legea erorii Monte Carlo**, cu o consecință practică dură: pentru o zecimală în plus avem nevoie de **100 de ori** mai multe repetări. Pe de altă parte, eroarea nu depinde de dimensiunea problemei, iar repetările sunt independente, deci se paralelizează perfect.

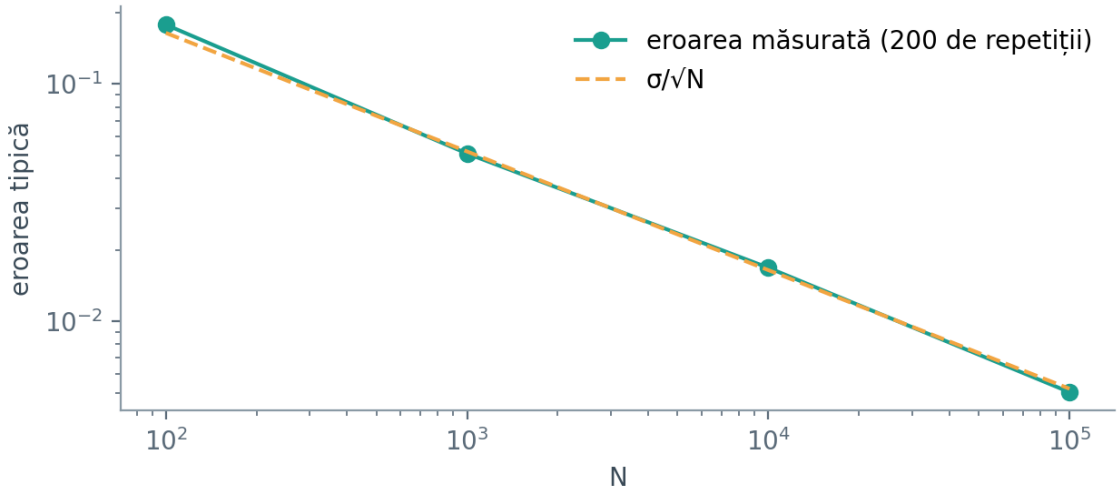


Figura 5. Eroarea tipică a estimării lui π în funcție de N (scări logaritmice): măsurători și legea σ/√N

3.3. Teorema limită centrală și intervalul de încredere

Legea numerelor mari garantează că media celor N rezultate converge către valoarea adevărată. **Teorema limită centrală** spune mai mult: pentru N mare, media are o distribuție aproximativ normală, cu abaterea standard σ/√N, oricare ar fi distribuția unui singur rezultat (dacă σ este finit). În Figura 6, 1 000 de estimări ale lui π cu câte 1 000 de puncte formează un clopot centrat în 3,142, cu abaterea standard 0,0513 (teorie: 0,0519) – deși fiecare punct contribuie doar cu 0 sau 4.

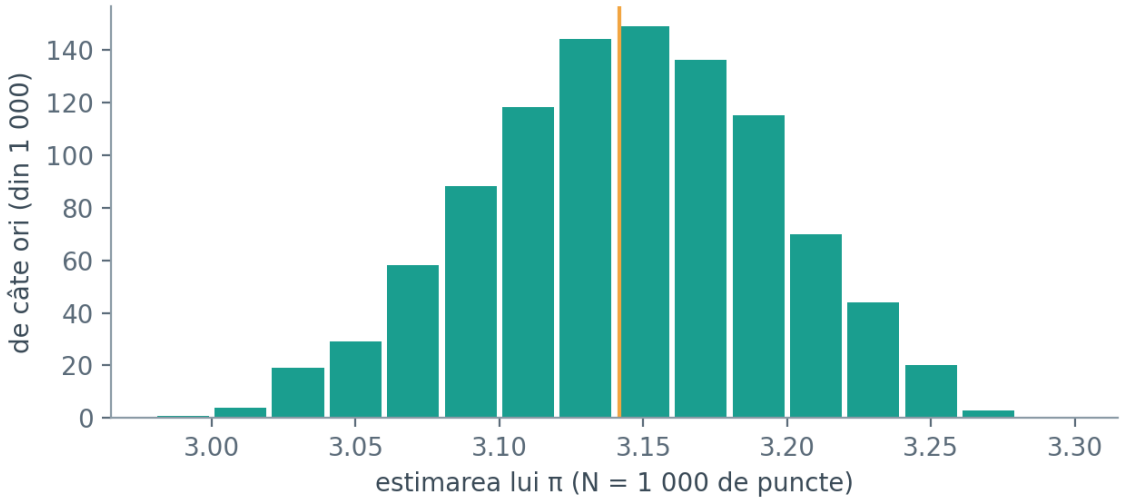


Figura 6. Histograma a 1 000 de estimări independente ale lui π, fiecare cu N = 1 000 de puncte; linia verticală marchează valoarea π

De aici obținem **intervalul de încredere de 95%**:

$$\bar{x} \pm 1{,}96 \cdot s / \sqrt{N}$$

unde s/√N este **eroarea standard** a mediei. Pentru π cu N = 10 000 am obținut x̄ = 3,1400, s = 1,6434, eroarea standard 0,0164 și semi-lățimea 0,0322, deci intervalul [3,108; 3,172], care conține π. Interpretarea corectă: dacă am repeta experimentul de multe ori, aproximativ 95% din intervalele construite astfel ar conține valoarea adevărată. Intervalul se îngustează ca 1/√N: de 4 ori mai multe repetări îl fac de 2 ori mai îngust. Pentru N mic (sub circa 30) se folosește distribuția t Student (săptămâna 6). Un rezultat se raportează sub forma „π ≈ 3,140 ± 0,032 (IÎ 95%, N = 10 000, seed 10)”.

Câte repetări ne trebuie? Din marja dorită E rezultă N ≈ (1,96·σ/E)², cu σ estimat dintr-o rulare pilot mică. Pentru π cu E = 0,01: σ ≈ 1,64, deci N ≈ 103 599. Pentru σ = 2 și E = 0,1 obținem (1,96·2/0,1)² ≈ 1 537.

De reținut

Eroarea Monte Carlo ≈ σ/√N; o zecimală în plus costă de 100 de ori mai multe repetări. Raportăm întotdeauna x̄ ± 1,96·s/√N. Numărul de repetări necesar: N ≈ (1,96σ/E)². Intervalul de 95% nu înseamnă „95% șanse ca valoarea să fie în acest interval anume”.

3.4. Integrare Monte Carlo și reducerea variației

Integrala unei funcții pe [a, b] este (b − a) înmulțit cu valoarea medie a funcției, pe care o estimăm din puncte uniforme: $\int_a^b f(x)dx \approx (b-a)\cdot(1/N)\Sigma f(x_i)$ (**metoda mediei**, Holban, cap. 9.1). Alternativa **hit-or-miss** aruncă puncte sub un dreptunghi și numără câte cad sub curbă. Pentru $\int_0^1 e^{(-x^2)}\,dx = 0{,}746824$ (fără primitivă

elementară), cu N = 10 000, metoda mediei a dat 0,7450 ± 0,0039, iar hit-or-miss 0,7423 ± 0,0086: la același N, metoda cu varianța mai mică este mai precisă. În una-două dimensiuni metodele clasice (trapeze, Simpson) sunt mult mai bune, dar o grilă cu 10 puncte pe axă în 10 dimensiuni are 10^{10} puncte, în timp ce eroarea Monte Carlo nu depinde de dimensiune – de aceea metoda domină în probleme cu multe variabile.

Tehnicile de **reducere a varianței** obțin aceeași precizie cu mai puține repetări: **variabilele antitetice** (perechi U și 1 − U, ale căror erori se compensează), **numerele aleatoare comune** (compararea variantelor cu aceleași numere aleatoare, săptămâna 6), **variabilele de control** și **eșantionarea prin importanță** (pentru evenimente rare). Pentru integrala de mai sus, 5 000 de perechi antitetice (tot 10 000 de evaluări) au dat 0,7470 ± 0,0008: varianța s-a redus de aproximativ 25 de ori.

3.5. Monte Carlo ca experiment statistic: riscul unui proiect

Un proiect eLearning are trei activități succesive, fiecare cu estimări optimistă, cea mai probabilă și pesimistă (Holban, tabelul 9.8): 4/6/8 zile, 5/7/9 zile și 2/4/6 zile. Planul naiv adună valorile cele mai probabile: 17 zile. Modelând fiecare activitate cu o distribuție triunghiulară (Holban folosește uniforma) și simulând 10 000 de proiecte, obținem durata medie 17,0 zile, P(durata ≤ 17) ≈ 50% și un termen cu 90% siguranță (P90) de 18,8 zile: planul naiv este, practic, o aruncare de monedă.

```
rng = np.random.default_rng(2024)
activitati = [(4, 6, 8), (5, 7, 9), (2, 4, 6)]   # (min, mod, max)
durata = sum(rng.triangular(a, m, b, 10_000) for a, m, b in activitati)
print(durata.mean(), (durata <= 17).mean(), np.percentile(durata, 90))
```

În realitate estimările sunt **asimetrice**: o activitate poate câștiga o zi, dar poate întârzia patru-cinci. Cu estimările 4/5/9, 5/6/11 și 2/3/7 zile, planul naiv este de 14 zile, dar simularea arată doar 3,8% șanse de a-l respecta, o durată medie de 17,3 zile, P80 ≈ 19 zile și P90 ≈ 20 zile (Figura 7). **Suma valorilor „cel mai probabil” nu este durata cea mai probabilă.**

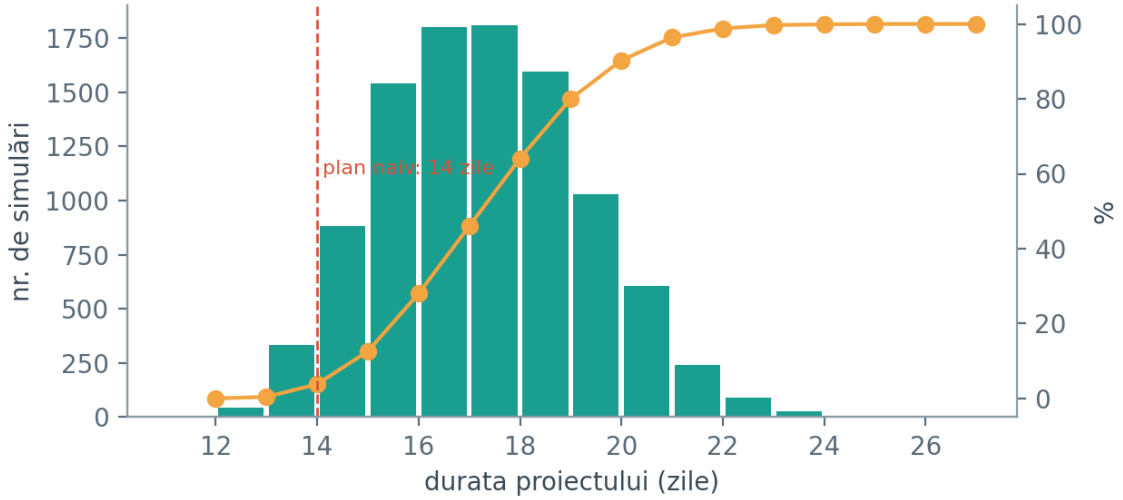


Figura 7. Distribuția duratei proiectului cu estimări asimetrice (10 000 de simulări) și probabilitatea cumulată de terminare

Pașii unei analize de risc Monte Carlo (Holban) sunt: identificăm variabilele de risc; le stabilim plajele de valori și distribuțiile; identificăm legăturile dintre ele (dependențele cresc riscul); simulăm și analizăm statistic; comunicăm **probabilități** (P50, P80, P90), nu un singur număr. Aceeași metodă se aplică la costuri, bugete, stocuri, capacitatea serverelor sau timpi de livrare.

3.6. Monte Carlo în informatică și greșeli frecvente

În grafica 3D, **path tracing** estimează culoarea fiecărui pixel din drumuri aleatoare ale razelor; imaginile cu puține raze pe pixel sunt „granulate”, iar pentru un zgomot de două ori mai mic trebuie de patru ori mai multe raze – legea σ/√N văzută pe ecran; motoarele moderne folosesc rețele neurale care reduc zgomotul. **Monte Carlo Tree Search** evaluează mutările prin partide simulate (AlphaGo). Metoda apare și în algoritmi randomizați (Miller–Rabin, quicksort cu pivot aleator), finanțe (riscul portofoliilor), fiabilitatea sistemelor distribuite și statistica bayesiană (MCMC, săptămâna 3).

Greșelile frecvente sunt: prea puține repetări, fără marja de eroare; rezultate fără interval de încredere; același seed în toate replicările; distribuții de intrare alese pentru comoditate; dependențe ignorate; confundarea mediei cu un termen sigur. Un test rapid: dublați N – dacă rezultatul se schimbă mai mult decât marja raportată, ceva nu este în regulă.

De reținut

Monte Carlo = model static stocastic evaluat de multe ori. Raportați media cu interval de încredere și percentilele relevante. Reducerea varianței poate economisi ordine de mărime de calcul. În analiza de risc, comunicați probabilități, nu un singur număr.

Glosar RO – EN

Română	English
experiment aleator	random experiment
variabilă aleatoare	random variable
funcție de masă de probabilitate	probability mass function (PMF)
densitate de probabilitate	probability density function (PDF)
funcție de repartiție	cumulative distribution function (CDF)
medie, valoare așteptată	mean, expected value
dispersie, abatere standard	variance, standard deviation
coeficient de variație	coefficient of variation
lipsa memoriei	memoryless property
proces Poisson	Poisson process
coadă grea	heavy tail
distribuție empirică	empirical distribution
eșantion, populație	sample, population
generator pseudo-aleator	pseudo-random number generator (PRNG)
generator congruențial liniar	linear congruential generator (LCG)
perioadă	period

Română	English
sămânță	seed
flux independent	independent stream
transformarea inversă	inverse transform method
acceptare – respingere	acceptance–rejection
legea numerelor mari	law of large numbers
teorema limită centrală	central limit theorem
eroare standard	standard error
interval de încredere	confidence interval
reducerea variației	variance reduction
variabile antitetice	antithetic variates
analiză de risc	risk analysis

Întrebări de autoevaluare

1. O variabilă ia valorile 1, 2, 3 cu probabilitățile 0,5, 0,3 și 0,2. Media ei este:

a) 2

b) 1,7

c) 1,5

d) 0,33

2. Ce distribuție folosim de obicei pentru timpul dintre două sosiri consecutive, când sosirile sunt independente și au rată constantă?

a) Normală

b) Uniformă

c) Exponențială

d) Binomială

3. Numărul de cereri pe secundă este Poisson cu $\lambda = 4$. Media și dispersia sunt:

a) 4 și 2

b) 4 și 16

c) 2 și 4

d) 4 și 4

4. Pentru o variabilă aleatoare continuă, $P(X = 20)$ este:

a) $f(20)$

b) 0

c) $F(20)$

d) $1 - F(20)$

5. O monedă corectă a căzut de 5 ori la rând „cap”. Probabilitatea ca a 6-a aruncare să fie tot „cap” este:

a) mai mică de 0,5

b) 0,5

c) mai mare de 0,5

d) 1/64

6. Proprietatea de lipsă a memoriei, $P(X > t + s \mid X > t) = P(X > s)$, o are dintre distribuțiile continue:

a) doar exponențiala

b) doar normala

c) toate distribuțiile continue

d) doar uniforma

7. Ce este un generator pseudo-aleator?

a) Un dispozitiv fizic care măsoară zgomotul termic

b) Un algoritm care produce numere imprevizibile, potrivit pentru parole

c) Un algoritm determinist care, pornind de la un seed, produce o secvență care trece testele statistice

d) O tabelă de numere publicată la Monte Carlo

8. O aplicație web trebuie să genereze token-uri de resetare a parolei. Ce folosim?

a) random.random() cu seed fix

b) numpy.random.default_rng()

c) modulul secrets

d) un LCG propriu

9. Un program apelează random.seed(42) în interiorul buclei de simulare, la fiecare pas. Ce se întâmplă?

a) Simularea devine mai precisă

b) Programul dă eroare

c) Rezultatele devin nereproductibile

d) Toate valorile „aleatoare” generate sunt identice

10. Cu tabelul de repartiție F = 0,2; 0,6; 0,8; 0,9; 1,0 (valorile 0, 1, 2, 3, 4), ce valoare generează transformarea inversă pentru U = 0,65?

a) 0

b) 1

c) 2

d) 3

11. O estimare Monte Carlo are eroarea ≈ 0,02 cu N = 10 000. Ce eroare aproximativă vom avea cu N = 1 000 000?

a) 0,0002

b) 0,002

c) 0,01

d) 0,2

12. Din N = 100 de rulări: $\bar{x}$ = 20 minute, s = 5 minute. Intervalul de încredere de 95% este:

a) 20 ± 5

b) 20 ± 9,8

c) 20 ± 0,98

d) 20 ± 0,5

13. Care este interpretarea corectă a unui interval de încredere de 95%?

a) Dacă repetăm experimentul de multe ori, ~95% din intervalele construite astfel conțin valoarea adevărată

b) Valoarea adevărată are 95% șanse să fie în acest interval anume

c) 95% din rezultatele individuale sunt în interval

d) Simularea este corectă în 95% din cazuri

14. De ce este metoda Monte Carlo avantajoasă pentru integrale în multe dimensiuni?

a) Dă rezultate exacte

b) Converge mai repede decât $1/\sqrt{N}$

c) Nu folosește numere aleatoare

d) Eroarea ei nu depinde de numărul de dimensiuni

15. Trei activități succesive au duratele cele mai probabile 5, 6 și 3 zile, dar pot întârzia mult mai mult decât pot câștiga. Planul de 14 zile este:

a) cel mai probabil rezultat

b) un termen sigur

c) probabil prea optimist

d) imposibil de evaluat

16. Vânzări săptămânale: 0, 1, 2, 3, 4 calculatoare cu probabilitățile 0,2; 0,4; 0,2; 0,1; 0,1. Care este media (calculatoare pe săptămână)?

(Răspuns numeric)

17. LCG cu a = 5, c = 3, m = 16 și $X_0 = 7$. Cât este X_1?

(Răspuns numeric)

18. Transformarea inversă pentru exponențială: $X = -\ln(U)/\lambda$, cu $\lambda = 2$ și $U = 0{,}5$. Cât este X (două zecimale)?

(Răspuns numeric)

19. O rulare pilot dă $\sigma = 2$. Câte repetări sunt necesare pentru o marjă de ± 0,1 la 95% încredere? ($N = (1{,}96 \cdot \sigma/E)^2$, rotunjit în sus)

(Răspuns numeric)

20. Din 1 000 de puncte aruncate uniform în pătratul unitate, 780 au căzut în sfertul de cerc de rază 1. Ce estimare a lui π obținem?

(Răspuns numeric)

Răspunsuri

1. b) – $1 \cdot 0{,}5 + 2 \cdot 0{,}3 + 3 \cdot 0{,}2 = 1{,}7$.

2. c) – Într-un proces Poisson, timpii dintre evenimente sunt exponențiali.

3. d) – La distribuția Poisson, media și dispersia sunt egale cu λ.

4. b) – Probabilitatea unei valori exacte este zero; au sens doar probabilitățile intervalelor.

5. b) – Aruncările sunt independente; moneda nu are memorie (eroarea jucătorului).

6. a) – Exponențiala este singura distribuție continuă fără memorie.

7. c) – PRNG = algoritm determinist; același seed dă aceeași secvență.

8. c) – Generatoarele pentru simulare sunt previzibile; pentru securitate se folosește un generator criptografic.

9. d) – Resetarea generatorului la fiecare pas produce mereu primul număr al secvenței.

10. c) – 0,65 cade în [0,6; 0,8), care corespunde valorii 2.

11. b) – N crește de 100 de ori, deci eroarea scade de $\sqrt{100} = 10$ ori.

12. c) – $1{,}96 \cdot 5/\sqrt{100} = 0{,}98$.

13. a) – Nivelul de încredere se referă la procedura de construcție a intervalului.

14. d) – Eroarea $\sigma/\sqrt{N}$ nu depinde de dimensiune, în timp ce o grilă crește exponențial.

15. c) – Cu distribuții asimetrice spre dreapta, suma valorilor celor mai probabile subestimează durata; în exemplul din curs avea ~4% șanse.

16. 1,5 – $0 \cdot 0{,}2 + 1 \cdot 0{,}4 + 2 \cdot 0{,}2 + 3 \cdot 0{,}1 + 4 \cdot 0{,}1 = 1{,}5$.

17. 6 – $(5 \cdot 7 + 3) \bmod 16 = 38 \bmod 16 = 6$.

18. 0,35 – $-\ln(0{,}5)/2 = 0{,}693/2 \approx 0{,}35$.

19. 1537 – $(1{,}96 \cdot 2/0{,}1)^2 = 39{,}2^2 = 1\,536{,}64 \rightarrow 1\,537$.

20. 3,12 – $\pi \approx 4 \cdot 780 / 1\,000 = 3{,}12$.

Bibliografie

- A. M. Law – Simulation Modeling and Analysis, ed. 6, McGraw Hill, 2024, cap. 6–8.
- S. M. Ross – Simulation, Academic Press – capitolele despre generarea variabilelor aleatoare.
- Ș. Holban – Tehnici de modelare și simulare, cap. 4 (statistică) și cap. 9 (Monte Carlo) (materiale de curs).
- R. J. Gibbens – Computer Systems Modelling, University of Cambridge – generarea numerelor și variabilelor aleatoare, intervale de încredere, reducerea variației.
- N. Metropolis, S. Ulam – The Monte Carlo Method, Journal of the American Statistical Association, 44(247), 1949.
- M. Matsumoto, T. Nishimura – Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator, ACM TOMACS, 1998.
- M. E. O'Neill – PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation, 2014.
- Documentația NumPy – Random Generator: numpy.org/doc/stable/reference/random.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5.